

Annotations :

Spring

=====

@Configuration - We can make any class configuration by adding @Configuration annotation. Whatever is mentioned within that will be managed by Spring. We can mention there whatever we want spring to Manage.

@Bean - Anything managed by spring is called Bean.

@Autowired - Autowiring is a spring way of performing dependency injection automatically by scanning for matching beans.

@Primary - Which beans should be used in case multiple beans gets qualified. @Primary annotated bean should be given preference when multiple beans are qualified.

@Qualifier - If we want any particular beans to be used by name then we can use @Qualifier. A specific bean should be autowired.

@Component - Help us to avoid creating @Bean for that class. Its annotation configuration. When we use @Bean and @Configuration, its Java configuration. When we do not have access to class to declare @Component, we create beans for that in configuration class. @component means, instance of that class will be managed by Spring. @Component is called stereotype annotation. @Component has some specialized annotation. Either we can use specialized annotation to give more idea to Spring about class or continue using stereotype.

Specialized annotation -

@Service (Business Logic)

@Controller (For restapi webapplication, controller class)

@Repository (indicates db operation class)

@ComponentScan("com.springtutorial.learn_spring_framework.game") - Ask spring to scan particular class to find @Component class.

@Lazy. - All beans gets initialized as soon as spring is loaded -- this is by default and its called Eager initialization. We can avoid initialization by using @Lazy annotation over the bean. It will be loaded only when its called by context. For example Dependency1 and Dependency2 example. We are getting not null because dependencies are already initialized.

@Scope(value = ConfigurableBeanFactory.SCOPE_SINGLETON) OR
@Scope(value = ConfigurableBeanFactory.SCOPE_PROTOTYPE) - Singleton : One object instance per Spring IOC container. Prototype - Multiple instance gets creteated per IOC container.

@PostConstruct — Jakarta Package. Can be mentioned only above method. Method gets executed as soon as spring initialize and all dependencies are wired in. Can have DB initialization code kind of thing.

@PreDestroy - Jakarta Package. method gets called just before spring destroys bean from the container

Jakarta CDI (Conext Dependency Injection) Annotations : -> Spring implements this specification/Interface... and also supports Jakarta annotations.

=====

@Inject (~ very similar to Spring @Autowired)

@Named (Similar to @component)

@Qualifier (same name in spring below 3).

@Scope

@Singleton

Spring Boot

=====

@SpringBootApplication — above main class. Main annotation, which includes @SpringBootConfiguration, @EnableAutoConfiguration, @ComponentScan. SpringBootConfiguration is same as configuration of normal spring project. We mention that thing on main class.

JPA/Jakarta/Hibernate (Over Model class - entity)

=====

@Entity(name = "table name"). — For mapping my bean to database table. Associating my bean directly with database table.

@Id - Marking any field primary key in the entity.

@Column(name="") - we can mark field on entity with this annotation if the name of field in the entity is different from name in the table.

@PersistentContext - Same as @Autowired, but special annotation for Spring JPA.

@Transactional - To make operation to happen in transaction. Required for JPA over Repository.

@GeneratedValue - To let know spring that this field is auto generated... either by auto increment or by sequence or any how.. There are parameters of this annotation to let spring know.

`@GeneratedValue(strategy = GenerationType.IDENTITY)`

@OneToOne - says that relation between Item and Item_detail is one to one.

This goes into model above field declaration. Field will be full other model and not particular field even if we are using particular field only as foreign key. Only annotation in itemdetail and mapped by mention in item table -

@OneToOne(mappedBy = "item", cascade = *CascadeType.ALL*, fetch = *FetchType.EAGER*). This relation ship is maintained by item detail.

@JoinColumn tells that Item_detail has item_id column which is foreign key for **Item.id** (item table's id column).

`@JoinColumn(name = "item_id", referencedColumnName = "id")`

Repository (ItemRepository extends JpaRepository<>)

=====

@Query - to write our own query. DataJPA provides many inbuilt methods. We can write our own query to perform some actions by using @Query(actual hibernate like query).

`@Query("UPDATE Item i SET i.name = :name WHERE i.id = :id")`

@Modifying - Write with @Query, to indicate that this is modifying database. Do not return anything but just return int which denotes affected rows.

@Transactional - can be used on method or at controller which is calling this method. By default it rolls back for only unchecked exception. Or mention `@Transactional(rollbackFor = Exception.class)`.

@Transactional(propagation =
Propagation.REQUIRES_NEW)

RestAPI Annotations (Controller)

=====

@RestController - to define controller which will have all Rest API paths.

Same as @Controller, but all methods return JSON/XML (via

@ResponseBody) automatically

@RequestMapping("/api/items") - Sets the base URL for all the handler methods in the controller. Base URL to be mentioned above the controller class.

@GetMapping('path'), @PostMapping, @PutMapping, @DeleteMapping - To define method/path as Get, Post, Put, Delete Path.

@PathVariable — <http://localhost/courses/{id}>... to get value of id in path we need pathvariable annotation.

@RequestParam - parameter in url -> <http://localhost/courses?id=5>.

@RequestParam("File") Multipart file — for uploading file . full file together.

@RequestBody — to request full body in the parameter of the method. This is useful in post request. Whatever we pass as a body in postman gets registered here in this variable. Like course object.

@ResponseStatus(HttpStatus.OK) — By default all api just returns 200 OK. To return particular class, we have to use @ResponseStatus.

@Valid - annotation is used in **Spring Boot and JPA** to trigger **bean validation**. It ensures that the incoming request data meets the **validation constraints** defined in the **entity/model class**.

@valid ONLY WORKS WITH @RequestBody. We have to mention @Valid in controller class without any parameter ahead of @RequestBody. As soon as request comes across @Valid, it checks entity/model class for what kind of validation needs to be applied. Like

@NotBlank("message"), **@Min/@Max**, **@Email**, **@Size(min,max)**. public Item createItems(@Valid @RequestBody Item item)

@Validated - Over controller class. This gives fine control over api call methods in controller class. Once @Validated is mentioned over controller class, validation annotation with @PathVariable and @RequestParam. We can use annotations such as @Min, @Max, @NotBlank, @NotNull, @Email, @Negative, @Assert etc... annotations AFTER @PathVariable or @RequestParam in the method but before actual variable.

public Optional<Item>
getItemByIDQueryParam(@RequestParam @Min(value = 3, message =

"Should be greater than 3") int id)

@RestControllerAdvice - Annotation to be used over the class which you want to be treated as global exception class. (its @ControllerAdvice special version which combines with @ResponseBody for RestAPI)

@ExceptionHandler(ConstraintViolationException.class) - Annotation to be used over the method in Global Exception. These method handles that particular exception.

public ResponseEntity<String>

handleConstraintViolationException(ConstraintViolationException ex). These global exception message is shown in the response.

@EnableAsync - Helps spring identify methods which are annotated with @Async annotation. Those methods gets processed asynchronously. This can be declared above controller class or application wide in main file.

@Async - We mention this in service class. we call async method in controller.

@CrossOrigin(origins = "<http://localhost:5173>"). — over the controller.. to allow request from only particular origins.

Spring AOP

=====

@Aspect - To be placed over the class which you want to be aspect class. Main logic of cross cutting concern goes here.

@Before, @After, @AfterThrowing, @AfterReturning, @Around - These are called advice.

@PointCut - Defines point where advice should be called.

@Pointcut("execution(* com.restapi.allannotations.controller.*(..))")